

PLATZI CONF BOGOTÁ · WORKSHOP A

Mientras llegan: clona el repo

```
git clone https://github.com/JairoTorregrosa/charlas  
cd charlas/2026-platzi-agent-sdk/taller  
uv sync
```

Necesitas `uv` y Claude Code con sesión iniciada (`claude` → `/login`).



repo del taller

github.com/JairoTorregrosa/charlas



Comunidad Claude
Colombia · WhatsApp

chat.whatsapp.com/LC3Zd1N8t6GJucYWKN5WkU

Crea tu primer agente con Claude Agent SDK

Un notebook, tres agentes, un primitivo a la vez



Jairo Torregrosa

Sr AI Research Engineer, Mercado Libre · Claude Ambassador

En 45 minutos

- 1 Qué es un agente y cómo funciona su loop.
- 2 Actuar el loop con personas: harness, LLM, tool, skill, MCP.
- 3 Los primitivos del Agent SDK, uno a la vez.
- 4 Correr tres agentes en tu laptop.

Dos capacidades convierten el modelo en agente



Cada sistema responde distinto

Revisa mis correos de la última semana y dime cuándo es mi próximo vuelo.

Un agente es un modelo más un harness

Modelo (LLM)

Recibe texto y devuelve texto. No puede tocar nada.

Harness

El programa alrededor: las tools, el loop que las llama, el contexto, los permisos.

Claude Code es un harness. El Agent SDK es ese mismo harness, como librería.

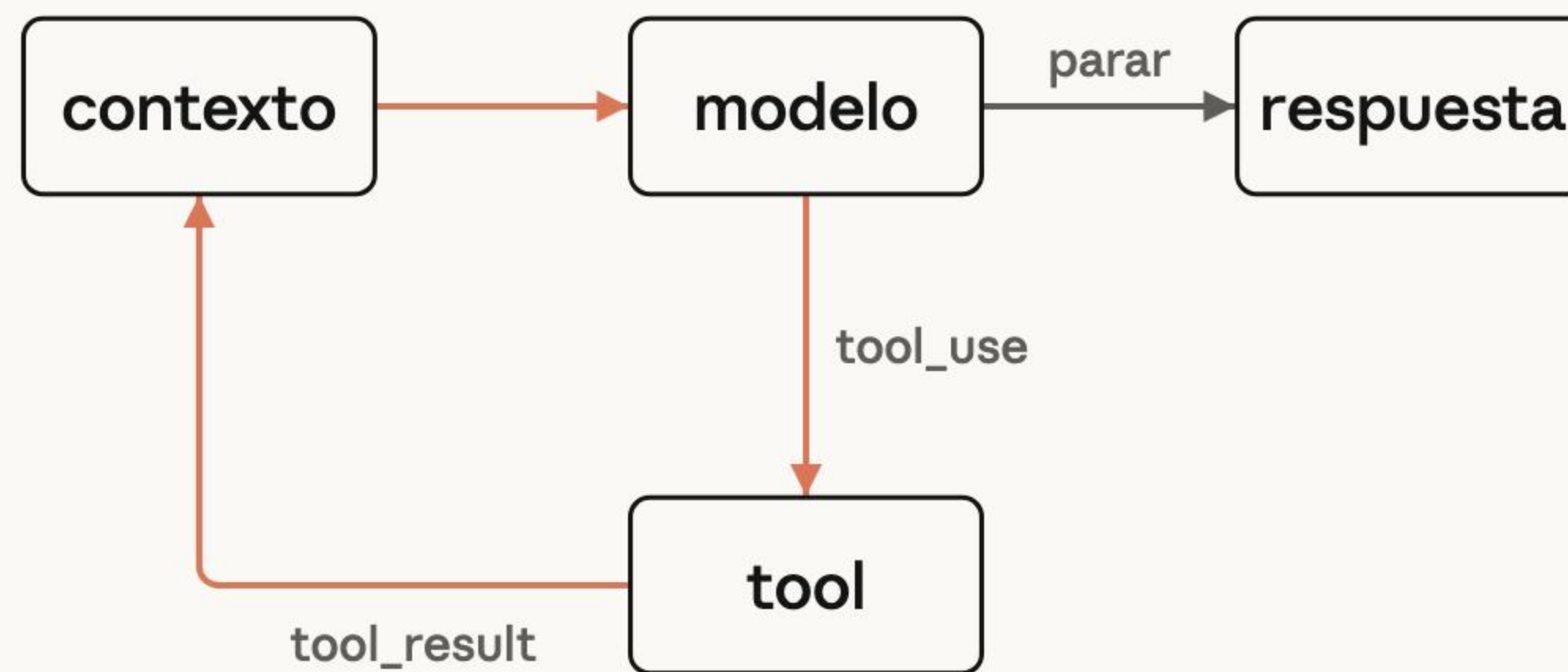
ANCLA 1

Un agente es un modelo que
usa tools dentro de un loop.

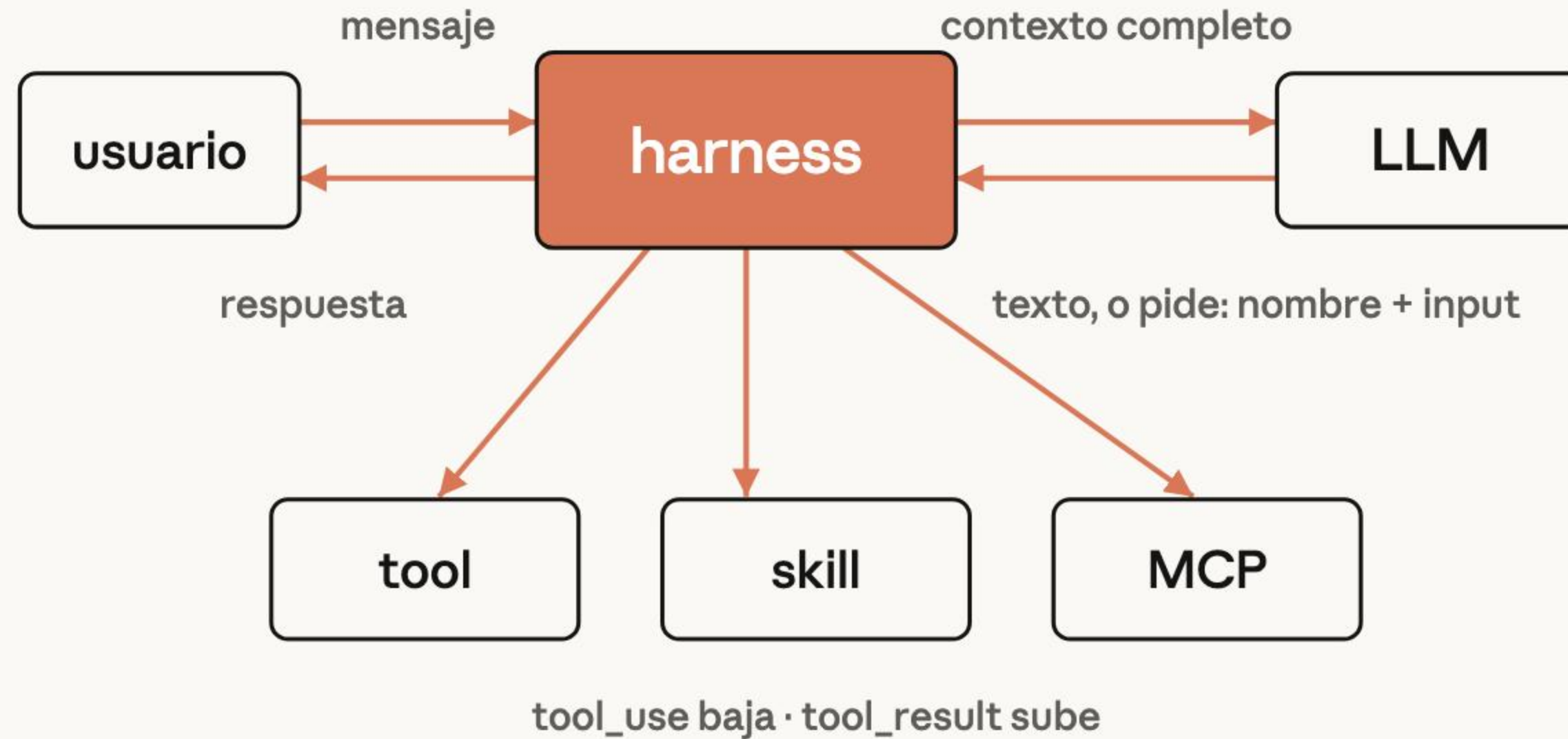
El agente repite hasta poder responder

```
mientras no termine:  
    respuesta = modelo(contexto)  
    si pide una tool:  
        contexto += ejecutar(tool)  
    si no:  
        entregar(respuesta)
```

Una vuelta del loop



Cinco papeles: el harness reparte el trabajo



Qué tiene cada uno en la mano

Papel	Tiene	Hace
harness	system prompt, catálogo (tools, skills, MCP) y contexto	arma el contexto, llama al LLM, reparte cada petición, anota cada resultado
LLM	nada: solo lo que el harness le lee	devuelve texto, o pide: nombre + input
tool	el reloj y la calculadora del computador	ejecuta literal (<code>hoy()</code>). Devuelve resultado o error
skill	un procedimiento escrito (<code>SKILL.md</code>)	lo lee completo cuando el LLM lo pide. Entra al contexto
MCP	el correo (Gmail)	responde como una tool más (<code>mcp__gmail__buscar</code> , <code>mcp__gmail__leer</code>)

Lo que el harness sabe antes del primer mensaje

```
# system prompt
Eres el asistente personal de Jairo. Respondes corto: fecha, hora y datos exactos.

# catálogo: solo nombres y descripciones
tools: hoy() – "la fecha de hoy"
skills: buscar-vuelo – "cómo encontrar el próximo vuelo en el correo"
mcp: gmail → mcp__gmail__buscar(query, desde) · mcp__gmail__leer(id)

# contexto: vacío. Llega el mensaje del usuario:
usuario: Revisa mis correos de la última semana y dime cuándo es mi próximo vuelo.
```

Actuemos el loop: cinco voluntarios

- 1 **harness** · el tablero es su contexto. Es el único que habla con el LLM.
- 2 **LLM** · de espaldas a la pantalla. Solo habla. Pide por nombre: `hoy()`.
- 3 **tool** · mira la pantalla. Responde `hoy()` tal cual.
- 4 **skill** · tiene el procedimiento. Lo lee en voz alta solo cuando el LLM lo pide.
- 5 **MCP** · tiene la bandeja de correos. Responde a `buscar` y `leer`.

El usuario es la sala: «Revisa mis correos de la última semana y dime cuándo es mi próximo vuelo.»

Lo que tiene el MCP: la bandeja de correos

c1 · 23 ago

De: Avianca
Asunto: 40 % de
descuento a Cartagena

Solo este fin de
semana. Compra ya.

c2 · 24 ago

De: Wingo
Asunto: Gracias por
volar con nosotros

Tu vuelo BOG → CTG
del 20 ago. Califica
tu experiencia.

c3 · 26 ago

De: LATAM
Asunto: Confirmación
de reserva ABC123

BOG → MDE
jue 3 sep, 06:15
Puerta por confirmar.

c4 · 27 ago

De: Booking
Asunto: Tu hotel en
Medellín

3 al 5 de septiembre.
Check-in desde 15:00.

`mcp__gmail__buscar(query, desde)` devuelve la lista (id, remitente, asunto, fecha) · `mcp__gmail__leer(id)` devuelve el correo completo

Lo que tienen la tool y la skill

tool · hoy()

`hoy()` → "sábado 29 de agosto de 2026"

Es lo único que sabe.

Sin esto, "última semana" y
"próximo" no significan nada.

skill · buscar-vuelo/SKILL.md

1. Pide la fecha de hoy.
2. Busca correos de aerolíneas de los últimos 7 días.
3. Ignora promociones y encuestas.
4. Lee el correo completo: fecha, hora, ruta, código de reserva.
5. Responde solo el vuelo más cercano en el futuro.

La skill entra al contexto cuando el LLM la pide con `Skill("buscar-vuelo")`. La tool responde en un segundo.

Una corrida posible (guía del harness)

```
turno 1 harness → LLM    system prompt + catálogo + pregunta
      LLM → harness      hoy()
      harness → tool     "sábado 29 de agosto de 2026"           se anota
turno 2 LLM → harness      Skill("buscar-vuelo")
      harness → skill     lee el procedimiento completo         se anota
turno 3 LLM → harness      mcp__gmail__buscar("vuelo reserva", desde="22 ago")
      harness → MCP       c1 Avianca · c2 Wingo · c3 LATAM · c4 Booking se anota
turno 4 LLM → harness      mcp__gmail__leer("c3")
      harness → MCP       "BOG → MDE, jue 3 sep, 06:15, reserva ABC123" se anota
turno 5 LLM → harness      "Tu próximo vuelo: jueves 3 de septiembre, 06:15,
      harness → usuario   Bogotá → Medellín, LATAM, reserva ABC123."
                        respuesta · result: 5 turnos
```

Reglas del juego

- 1 Todo pasa por el harness. Nadie más habla con el LLM.
- 2 El LLM pide por nombre, con input exacto: `mcp__gmail__leer("c3")`. No mira la pantalla.
- 3 Tool, skill y MCP responden literal. No opinan.
- 4 Cada resultado se anota en el tablero. Lo que no está anotado, el LLM no lo sabe.
- 5 Algo que no existe → `is_error`. El LLM reintenta.
- 6 El LLM responde sin pedir nada → el harness anuncia `result: N turnos`.

En el SDK, cada papel es un mensaje

- 1 **SystemMessage** `init` · con qué arrancó el harness: modelo, cwd, catálogo.
- 2 **AssistantMessage** · el LLM habla, o pide algo (`ToolUseBlock`).
- 3 **UserMessage** · la tool, la skill o el MCP responden (`ToolResultBlock` , a veces `is_error`).
- 4 **ResultMessage** · el recibo: turnos, costo, `session_id` .

Esto es lo que entrega `query()` , uno por uno.

ANCLA 2

El LLM solo habla. El
harness reparte, anota
y decide cuándo parar.

EL ENIGMA

Todo esto ya lo hace Claude
Code en tu terminal.
Entonces, ¿para qué un SDK?

Los primitivos, uno por celda

#	Primitivo	Qué le da al agente
1	<code>query()</code> + <code>system_prompt</code>	una personalidad y una vuelta
2	<code>cwd</code> + <code>tools</code> nativas	manos: leer, correr comandos
3	el stream de mensajes	ver cada paso del ciclo
4	<code>setting_sources</code> + <code>skill</code>	un procedimiento que carga cuando hace falta
5	hooks	un portero que dice no
6	sub-agentes	ayudantes que trabajan en paralelo
7	MCP + <code>max_budget_usd</code>	herramientas de otros, con tope de gasto
8	<code>resume</code> · dos <code>query()</code> a la vez	seguir la conversación · orquestar desde código

1·query() + system_prompt: una personalidad, una vuelta

```
from claude_agent_sdk import query, ClaudeAgentOptions

opciones = ClaudeAgentOptions(
    model="claude-sonnet-5",
    system_prompt=PERSONAJE,      # quién es: El Parjero Crítico
    tools=[],                     # sin herramientas
    max_turns=1,                 # una sola vuelta
)

async for mensaje in query(prompt=CODIGO, options=opciones):
    ...
```

Con `max_turns=1` y `tools=[]` esto es un chat. Todavía no es un agente.

2. cwd + tools nativas: manos

```
opciones = ClaudeAgentOptions(  
    model="claude-sonnet-5",  
    system_prompt=PERSONAJE,  
    cwd=str(WS),           # su casa: workspace_parzero/  
    allowed_tools=["Bash"], # una sola mano, sin preguntar  
    setting_sources=[],     # no leas config de disco todavía  
    max_turns=10,  
)  
  
# prompt: "https://github.com/JairoTorregrosa/crea-tu-web"  
# [usa] Bash: git clone https://github.com/... repos/crea-tu-web  
# [usa] Bash: find repos/crea-tu-web -name "*.rs" | wc -l
```

Cada [usa] es una vuelta: el modelo pide, el harness ejecuta, el resultado vuelve. Eso es un agente.

4 · La casa del agente es su configuración

```
workspace_parzero/  
├── .claude/  
│   ├── skills/roast-de-codigo/SKILL.md    ← el procedimiento  
│   └── agents/inspector.md                ← el ayudante  
├── repos/                                  ← donde clona  
└── roast.md                               ← lo único que escribe
```

```
opciones = ClaudeAgentOptions(cwd=str(WS), setting_sources=["project"], ...)
```

Un skill es una carpeta con un `SKILL.md`. El modelo ve solo su nombre; el texto entra cuando lo usa.

5· Hooks: un portero que dice no

```
async def solo_roast(input_data, tool_use_id, context):
    if Path(input_data["tool_input"]["file_path"]).name == "roast.md":
        return {} # pasa
    return {"hookSpecificOutput": {
        "hookEventName": "PreToolUse",
        "permissionDecision": "deny",
        "permissionDecisionReason": "Solo puedes escribir roast.md."}}

hooks={"PreToolUse": [HookMatcher(matcher="Write|Edit", hooks=[solo_roast])]}
```

El motivo se lo lee el modelo como resultado: no se cae, busca otro camino.

6 · Sub-agentes: ayudantes en paralelo

```
# workspace_parzero/.claude/agents/inspector.md → un agente más pequeño
allowed_tools=["Read", "Glob", "Grep", "Agent"] # Agent: puede lanzar ayudantes

# "Lanza un inspector por cada archivo, todos a la vez."
[parzero] usa Agent: inspeccionar cache.rs
[parzero] usa Agent: inspeccionar lists.rs
  [inspector] usa Read: repos/crea-tu-web/crates/core/src/cache.rs
  [inspector] usa Grep: unwrap(
```

El ayudante trabaja aparte y devuelve solo su respuesta. Sus mensajes traen `parent_tool_use_id`.

7·MCP y max_budget_usd: herramientas de otros, con tope

```
opciones = ClaudeAgentOptions(  
    cwd=str(WSV), setting_sources=["project"],      # carga workspace_vivienda/.mcp.json  
    allowed_tools=["mcp__exa__*", "mcp__firecrawl__*", "WebFetch"],  
    max_turns=8,  
    max_budget_usd=0.15,                          # tope duro en dólares  
)  
# tools de un servidor MCP: mcp__<servidor>__<tool>  
# tope alcanzado → ResultMessage subtype="error_max_budget_usd"
```

MCP es la forma estándar de darle herramientas a un modelo.
Un servidor MCP es una caja con tools que alguien más hizo.

ANCLA 3

Tú no escribes el loop.
Escribes el personaje,
la casa y los porteros.

Abre el notebook

```
cd charlas/2026-platzi-agent-sdk/taller  
uv sync  
uv run jupyter lab demo.ipynb
```

```
# la celda 0 revisa: claude --version · ANTHROPIC_API_KEY no está
```

Sin API key: el SDK usa tu sesión de Claude Code. En el notebook se escribe `await`, nunca `asyncio.run()`.

Tres agentes, ninguno con tools propias

El Parcerero Crítico

Clona un repo público y lo roastea en `roast.md`. Skill, ayudante y porteros.

Buscador de vivienda

Avisos reales en Bogotá por MCP (Exa, Firecrawl). Tabla en `candidatos.md`. `resume`.

El Monitor

Te explica un paper en cinco preguntas. MCP alphaXiv. `explicacion.md`.

Cada agente = `agent_<x>.py` + `workspace_<x>/`.

Cada uno escribe un solo archivo; un hook lo garantiza.

El recorrido del notebook

-
- 3 **El Parcero Crítico** · 3.1 personaje → 3.2 manos → 3.3 stream → 3.4 skill → 3.5 porteros → 3.6 ayudantes → 3.7 streaming → 3.8 `agent_parcero.py`
 - 4 **Buscador de vivienda** · 4.1 MCP + `max_budget_usd` → 4.2 `resume`
 - 5 **El Monitor** · 5.1 un servidor MCP externo → 5.2 `agent_monitor.py`
 - 6 **El enigma, resuelto** · dos `query()` a la vez con `asyncio.gather`
-

En sala: 3.1 → 3.5. El resto, esta noche.

Corre y mira el stream

```
[init] modelo=claude-sonnet-5 · cwd=.../workspace_parcer0 · tools=['Bash']  
[modelo pide] Bash {'command': 'git clone https://github.com/JairoTorregrosa/crea-tu-web'}  
[tool responde] 312 caracteres  
[modelo pide] Bash {'command': 'find repos/crea-tu-web -type f | wc -l'}  
[tool responde] 3 caracteres  
[modelo dice] 63 archivos. Ya vi suficiente para empezar a sufrir.  
[fin] 3 vueltas · USD 0.041
```

Las mismas líneas que el harness anotó en el tablero.

Haz el tuyo

- 1 Copia `agent_parcerero.py` y `workspace_parcerero/`. Cámbiales el nombre.
- 2 Cambia el personaje (`PERSONAJE`), el skill (`SKILL.md`) y el ayudante (`.claude/agents/`).
- 3 Deja los porteros: un agente que solo escribe un archivo no te daña nada.
- 4 Pon siempre `max_turns` y `max_budget_usd`.

Cheat sheets en `taller/docs/`: eventos, opciones, tools, workspace, permisos y hooks, errores, proveedores.

¿Para qué un SDK, si Claude Code ya hace todo esto?

- 1 **Lo que probaste a mano, lo pones a correr solo.** Un cron, un CI, un bot: `query()` en un script.
- 2 **Quieres el cerebro de Claude Code dentro de tu producto.** El Parcero es un producto; le pones una interfaz y ya.
- 3 **Quieres armar agentes desde código.** Prompts con datos, dos agentes en paralelo, un portero con tus reglas.

PLATZI CONF BOGOTÁ 2026

Ya puedes construir tu agente

Tú escribes el personaje, la casa y
los porteros. El loop lo pone el SDK.

Gracias a Platzi por el espacio.



repo del taller
github.com/JairoTorregrosa/charlas



Comunidad Claude
Colombia · WhatsApp
chat.whatsapp.com/LC3Zd1N8t6GJucYWKN5WkU